

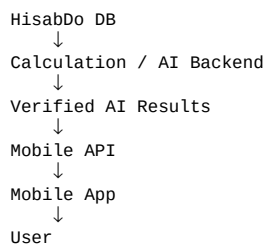
HisabDo AI — Day 17

Mobile Integration — Detailed Requirements, API Flow, Screens, Testing & Acceptance

Goal: Connect the verified HisabDo AI backend capabilities to the mobile application so users can view Financial Health, Recommendations, Spending Patterns, Anomalies, Cash-Flow Forecasts, Budget Recommendations and Ask HisabDo AI from one consistent mobile experience.

Architecture rule: Mobile is a presentation/client layer. It must not recalculate financial metrics or invent AI content. Backend services remain the source of truth.

End-to-End Flow



1. Define and finalize Day 17 Mobile Integration requirements

The mobile app must securely retrieve verified AI results from the backend and present them in simple, readable screens. MVP integration covers one authenticated user at a time, user-isolated API responses, loading/error/empty states, pull-to-refresh or equivalent refresh, and a complete navigation path from dashboard to detailed AI results and Ask HisabDo AI.

2. Define mobile AI feature scope

The Day 17 mobile scope contains: AI Dashboard, Financial Health, Recommendations, Spending Patterns, Anomalies, Cash-Flow Forecast, Budget Recommendations and Ask HisabDo AI. AI explanations must be grounded in verified backend results. Mobile does not calculate source-of-truth financial values.

3. Define mobile screens required for HisabDo AI

Recommended screens: (1) AI Dashboard, (2) Financial Health Detail, (3) Recommendations List, (4) Recommendation Detail, (5) Spending Patterns, (6) Anomalies, (7) Cash-Flow Forecast, (8) Budget Recommendations, and (9) Ask HisabDo AI. Each screen requires Loading, Loaded, Empty and Error states where applicable.

4. Define API-to-mobile integration flow

The authenticated mobile app calls the mobile AI API. The API obtains user identity from the authenticated session/token, not from a userId supplied by the app. Backend services retrieve verified results and return DTOs designed for mobile. The app maps DTOs to UI models and renders them. A refresh repeats the request and replaces stale data with the latest successful response.

5. Define recommendation display structure

Recommendation cards should show Title, Category, Priority, Severity when available, Why it was generated, Financial Pattern, Suggested Action, Expected Benefit only when verified, and Limitation when supplied. High-priority recommendations should be visually prominent, but the app must not change backend priority.

6. Define Financial Health display

Show the 0–100 score, classification, key verified factors, and a simple explanation. Recommended sections: Health Score, Saving Rate, Expense Ratio, Budget Utilization, Cash Flow, Expense Growth and AI Explanation. If a factor is unavailable, show “Not available” rather than estimating it.

7. Define Spending Pattern display

Show category, current amount, previous amount, percentage change, trend, unusual flag and recurring flag when supplied. Useful mobile presentation: category cards/list, trend indicator and short explanation. Do not derive a new trend on the mobile side.

8. Define Anomaly display

Show anomaly type, category, severity, amount and verified explanation. Provide a clear review action such as "Review transaction". Do not label an anomaly as fraud, theft or unauthorized activity unless the backend explicitly supplies such a verified classification.

9. Define Cash-Flow Forecast display

Show expected income, expected expense, expected saving, expected closing balance only when available, confidence and limitations. Forecast values are read-only verified results. If the backend marks the forecast insufficient, show the limitation instead of fabricating a forecast.

10. Define Budget Recommendation display

Show category, historical average, existing budget when available, recommended budget, variance when available, reason and data sufficiency. If no existing budget exists, do not display an invented budget comparison.

11. Define Ask HisabDo AI mobile flow

User opens Ask HisabDo AI → enters a question → app validates non-empty input → shows loading → POSTs the question to the backend → backend identifies intent and retrieves verified data → AI explanation is generated → response returns with verification/limitation metadata → app displays answer. Example question: "Why is my saving rate low?"

12. Define loading, error and empty-data handling

Loading: show skeleton/spinner and disable duplicate submission. Error: show a short retryable message and preserve previously loaded data where appropriate. Empty: explain that no verified data is currently available and do not treat the condition as a system failure. Offline/network error: show retry. Authentication error: require re-authentication. Timeout: offer retry.

13. Coordinate Jaffer, Omesha, Aarti and Laiba

Jaffer owns backend API contracts, authentication/user isolation and integration with calculation services. Omesha owns AI/ML result contracts, intent/data mapping and verified analytics. Aarti owns recommendation/AI explanation structure, prompt grounding and no-invention rules. Laiba owns mobile screens, API client integration, state handling and end-to-end mobile testing. Taha coordinates requirements, reviews and final acceptance.

14. Review backend API integration

Confirm endpoint paths, HTTP methods, authentication, DTO schemas, nullable fields, error codes, pagination if needed, timeout behavior and versioning. Recommended MVP endpoints are GET /api/mobile/ai/dashboard and POST /api/mobile/ai/ask. Existing HisabDo endpoints can be used instead if they already satisfy the contract.

15. Review AI response integration

The mobile client should receive a structured Ask HisabDo response containing answer, intent, verification status and limitation. The UI must preserve limitation text and must not modify financial facts. Recommendation expected benefits must only appear when the backend marks them verified.

16. Validate complete mobile user flow

Test login/authentication → AI Dashboard → Financial Health → Recommendation → Spending Pattern → Anomaly → Cash-Flow Forecast → Budget Recommendation → Ask HisabDo AI → refresh → logout/re-login. Verify that data belongs to the authenticated user and that each screen behaves correctly with complete, partial, empty and error data.

17. Testing and final Day 17 acceptance

Acceptance requires successful API integration, correct DTO mapping, user isolation, all required screens, all four state types, AI response rendering, no mobile-side financial calculations, correct verified-status handling, retry behavior, and

end-to-end testing on representative datasets. Final sign-off should record API, UI, AI and security test results.

Mobile Screen Specification

Screen	Main content	Empty state
AI Dashboard	Health, top recommendations, patterns, anomalies, forecast, budgets	No AI insights available yet
Financial Health	Score, classification, factors, explanation	Health data unavailable
Recommendations	Title, priority, severity, reason, action	No recommendations available
Recommendation Detail	Full explanation, pattern, action, limitation	Recommendation unavailable
Spending Patterns	Category trends and verified flags	No spending pattern data
Anomalies	Type, category, severity, amount, explanation	No anomalies detected/available
Cash-Flow Forecast	Income, expense, saving, balance, confidence	Forecast unavailable/insufficient
Budget Recommendations	Historical average, budget, recommendation, reason	No budget recommendation data
Ask HisabDo AI	Question, answer, verification, limitation	No answer until question is entered

API Contract

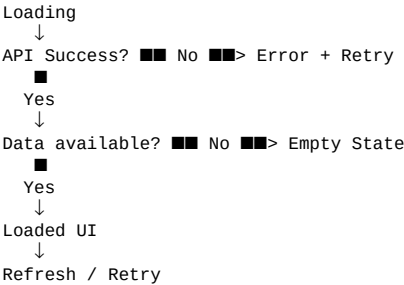
```
GET /api/mobile/ai/dashboard
Authorization: Bearer <token>

POST /api/mobile/ai/ask
Content-Type: application/json

{
  "question": "Why is my saving rate low?"
}

Response:
{
  "answer": "...",
  "intent": "FinancialHealth",
  "isVerified": true,
  "limitation": null
}
```

State Management



Team Responsibility Matrix

Person	Day 17 responsibility
Taha	Requirements, coordination, reviews and final acceptance
Jaffer	Backend API, authentication, user isolation and service integration
Omesha	AI/ML result contracts, intent/data mapping and verified analytics
Aarti	AI explanation structure, prompt grounding and response rules
Laiba	Mobile screens, API client, state handling and E2E mobile testing

Security Requirements

- User ID must come from authenticated claims/session.
- Mobile must never be able to request another user's AI data by changing a userId parameter.
- Backend authorization must be checked on every AI data endpoint.
- Do not store sensitive financial responses in logs unnecessarily.

- AI responses should retain verification and limitation metadata.
- Mobile should use HTTPS in production.
- Do not calculate or modify source-of-truth financial values in the client.

Testing Matrix

Area	Test
Authentication	Valid token, expired token, missing token
User isolation	User A cannot retrieve User B data
Dashboard	All widgets populated
Partial data	One AI module unavailable while others load
Empty data	No transactions / no budget / no forecast
Recommendations	Priority, severity and action render correctly
Financial Health	Score and factors match API
Patterns	Trend and verified flags match API
Anomalies	Severity and explanation match API
Forecast	Confidence and limitation shown
Budget	Existing budget nullable handling
Ask AI	Valid question, empty question, long question, timeout
Network	Retry and offline handling
Refresh	Latest successful response replaces stale data
Security	No client-supplied userId trust

Final Day 17 Acceptance Checklist

- ✓ All 17 requested tasks are covered.
- ✓ Mobile AI feature scope finalized.
- ✓ Required screens and screen states defined.
- ✓ API-to-mobile flow documented.
- ✓ Recommendation, Health, Pattern, Anomaly, Forecast and Budget displays defined.
- ✓ Ask HisabDo AI flow defined.
- ✓ Loading, error and empty handling defined.
- ✓ Jaffer/Omesha/Aarti/Laiba responsibilities defined.
- ✓ Backend API and AI response review criteria defined.
- ✓ Complete user flow and security validation defined.
- ✓ Testing matrix defined.
- ✓ .NET reference implementation supplied.
- ✓ Mobile remains a presentation layer; backend remains source of truth.

Reference .NET Code

```
// HisabDo AI - Day 17 Mobile Integration
// .NET 8 / ASP.NET Core reference API contracts and integration layer.
// Source of truth: HisabDo backend. Mobile renders verified DTOs only.

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using System.ComponentModel.DataAnnotations;

namespace HisabDo.AI.MobileIntegration;

public sealed record FinancialHealthDto(
    double Score, string Classification, double SavingRate,
    double ExpenseRatio, double BudgetUtilization,
    double CashFlow, double ExpenseGrowthPercent);

public sealed record SpendingPatternDto(
    string Category, double CurrentAmount, double PreviousAmount,
    double ChangePercent, string Trend, bool IsUnusual, bool IsRecurring);

public sealed record AnomalyDto(
    string Type, string Category, string Severity, double Amount, string Explanation);

public sealed record CashFlowForecastDto(
    double ExpectedIncome, double ExpectedExpense, double ExpectedSaving,
    double? ExpectedClosingBalance, string Confidence, string Limitation);

public sealed record BudgetRecommendationDto(
    string Category, double HistoricalAverage, double? ExistingBudget,
    double RecommendedBudget, double? BudgetVariance, string Reason,
    string DataSufficiency);

public sealed record RecommendationDto(
    string RecommendationId, string Category, string Title, string Reason,
    string FinancialPattern, string Priority, string Severity,
    string SuggestedAction, string? ExpectedBenefit,
    bool ExpectedBenefitVerified, string? Limitation);

public sealed record AskHisabDoRequest([property: Required] string Question);

public sealed record AskHisabDoResponse(
    string Answer, string Intent, bool IsVerified, string? Limitation);

public sealed record DashboardResponse(
    FinancialHealthDto? FinancialHealth,
    IReadOnlyList<RecommendationDto> Recommendations,
    IReadOnlyList<SpendingPatternDto> SpendingPatterns,
    IReadOnlyList<AnomalyDto> Anomalies,
    CashFlowForecastDto? CashFlowForecast,
    IReadOnlyList<BudgetRecommendationDto> BudgetRecommendations);

public sealed record ApiError(string Code, string Message);

public interface IFinancialHealthService
{
    Task<FinancialHealthDto?> GetAsync(string userId, CancellationToken ct);
}

public interface ISpendingPatternService
{
    Task<IReadOnlyList<SpendingPatternDto>> GetAsync(string userId, CancellationToken ct);
}

public interface IAnomalyService
{
    Task<IReadOnlyList<AnomalyDto>> GetAsync(string userId, CancellationToken ct);
}

public interface ICashFlowForecastService
{
    Task<CashFlowForecastDto?> GetAsync(string userId, CancellationToken ct);
}

public interface IBudgetRecommendationService
{
    Task<IReadOnlyList<BudgetRecommendationDto>> GetAsync(string userId, CancellationToken ct);
}

public interface IRecommendationService
{
    Task<IReadOnlyList<RecommendationDto>> GetAsync(string userId, CancellationToken ct);
}

public interface IAskHisabDoService
{
    Task<AskHisabDoResponse> AskAsync(
        string userId, string question, CancellationToken ct);
}

public interface IMobileDashboardService
{
    Task<DashboardResponse> GetDashboardAsync(
        string userId, CancellationToken ct);
}

public sealed class MobileDashboardService : IMobileDashboardService
```

```

{
    private readonly IFinancialHealthService _health;
    private readonly IRecommendationService _recommendations;
    private readonly ISpendingPatternService _patterns;
    private readonly IAnomalyService _anomalies;
    private readonly ICashFlowForecastService _forecast;
    private readonly IBudgetRecommendationService _budgets;

    public MobileDashboardService(
        IFinancialHealthService health,
        IRecommendationService recommendations,
        ISpendingPatternService patterns,
        IAnomalyService anomalies,
        ICashFlowForecastService forecast,
        IBudgetRecommendationService budgets)
    {
        _health = health;
        _recommendations = recommendations;
        _patterns = patterns;
        _anomalies = anomalies;
        _forecast = forecast;
        _budgets = budgets;
    }

    public async Task<DashboardResponse> GetDashboardAsync(
        string userId, CancellationToken ct)
    {
        if (string.IsNullOrEmpty(userId))
            throw new ArgumentException("Authenticated user is required.");

        var healthTask = _health.GetAsync(userId, ct);
        var recommendationTask = _recommendations.GetAsync(userId, ct);
        var patternTask = _patterns.GetAsync(userId, ct);
        var anomalyTask = _anomalies.GetAsync(userId, ct);
        var forecastTask = _forecast.GetAsync(userId, ct);
        var budgetTask = _budgets.GetAsync(userId, ct);

        await Task.WhenAll(
            healthTask, recommendationTask, patternTask,
            anomalyTask, forecastTask, budgetTask);

        return new DashboardResponse(
            await healthTask,
            await recommendationTask,
            await patternTask,
            await anomalyTask,
            await forecastTask,
            await budgetTask);
    }
}

[ApiController]
[Authorize]
[Route("api/mobile/ai")]
public sealed class MobileAiController : ControllerBase
{
    private readonly IMobileDashboardService _dashboard;
    private readonly IAskHisabDoService _ask;

    public MobileAiController(
        IMobileDashboardService dashboard,
        IAskHisabDoService ask)
    {
        _dashboard = dashboard;
        _ask = ask;
    }

    [HttpGet("dashboard")]
    public async Task<ActionResult<DashboardResponse>> GetDashboard(
        CancellationToken ct)
    {
        // IMPORTANT: userId must come from the authenticated identity,
        // never from a mobile query/body parameter.
        var userId = User.FindFirst("sub")?.Value;

        if (string.IsNullOrEmpty(userId))
            return Unauthorized(new ApiError(
                "AUTHENTICATION_REQUIRED",
                "Authenticated user context is required."));

        return Ok(await _dashboard.GetDashboardAsync(userId, ct));
    }

    [HttpPost("ask")]
    public async Task<ActionResult<AskHisabDoResponse>> Ask(
        [FromBody] AskHisabDoRequest request,
        CancellationToken ct)
    {
        var userId = User.FindFirst("sub")?.Value;

        if (string.IsNullOrEmpty(userId))

```

```

        return Unauthorized();

    if (request == null || string.IsNullOrEmpty(request.Question))
        return BadRequest(new ApiError(
            "EMPTY_QUESTION", "Please enter a financial question."));

    if (request.Question.Length > 1000)
        return BadRequest(new ApiError(
            "QUESTION_TOO_LONG",
            "Question must be 1000 characters or less."));

    return Ok(await _ask.AskAsync(
        userId, request.Question.Trim(), ct));
}

public enum MobileLoadState
{
    Loading, Loaded, Empty, Error
}

public sealed record MobileScreenState<T>(
    MobileLoadState State, T? Data,
    string? UserMessage, string? ErrorCode);

/*
Mobile API contract:

GET /api/mobile/ai/dashboard
POST /api/mobile/ai/ask

Ask request:
{
    "question": "Why is my saving rate low?"
}

Ask response:
{
    "answer": "...",
    "intent": "FinancialHealth",
    "isVerified": true,
    "limitation": null
}

Recommended mobile navigation:
AI Dashboard
- Financial Health
- Recommendations
- Spending Patterns
- Anomalies
- Cash-Flow Forecast
- Budget Recommendations

Ask HisabDo AI
- Question input
- Loading
- Verified AI answer
- Limitation/source status

Mobile rules:
- Never calculate financial values in the app.
- Never trust a user-supplied userId.
- Show ExpectedBenefit only when ExpectedBenefitVerified=true.
- Show verified AI status only when IsVerified=true.
- Empty arrays are valid empty states, not errors.
*/

```